Team G Coding Standards

Purpose: Establish practices for creating readable, maintainable, and reliable code

1. Naming and Formatting

- Use PascalCase for React component names and component filenames (e.g., UserProfile.jsx).
- Use camelCase for variables, functions, props, and custom Hooks (e.g., handleSubmit, useUserData).
- Use const by default and let when reassignment is necessary. Do not use var.
- Use two-space indentation, semicolons, and single quotes for JavaScript strings, unless the project's formatter specifies otherwise.
- Place imports at the top of each file, with external dependencies before internal modules. Remove unused imports and variables.

2. Components and JSX

- Use functional components and Hooks for all new React components. Use one primary component per file.
- Give each component one primary responsibility. Extract shared UI into reusable components when it appears in multiple locations.
- Use descriptive prop names and destructure props in component parameters when practical.
- Use semantic HTML elements where appropriate. Give every rendered list item a stable, unique key; do not use array indices when items can be reordered, added, or removed.
- Keep JSX readable with consistent indentation and multiline formatting for complex elements.

3. State and Hooks

- Follow the official Rules of React. Call Hooks only at the top level of React components or custom Hooks.
- Never mutate props or state directly. Use state setters and immutable updates.
- Keep rendering pure. Do not perform network requests, modify external state, or trigger side effects during rendering.

- Use event handlers for user-triggered actions and useEffect to synchronize with external systems. Do not use effects for values that can be calculated during rendering.
- Avoid redundant state. Derive values from existing props and state whenever possible.
- Use useMemo and useCallback only when needed for measured performance improvements or specific dependency requirements.

4. Code Quality and Accessibility

- Keep functions focused and avoid duplicating logic. Extract shared logic into helper functions or custom Hooks when appropriate.
- Handle errors and asynchronous operations explicitly. Provide loading and error states for user-facing asynchronous operations when applicable.
- Use accessible labels for form controls, meaningful alternative text for informative images, and descriptive text for interactive controls.
- Do not commit credentials, API keys, or other secrets. Use the project's approved configuration practices.
- Remove debugging statements, unused code, and temporary workarounds before submitting changes.

5. Testing and Team Workflow

- Add or update tests when changing behavior. Test expected outcomes, relevant error conditions, and important edge cases.
- Run available tests, linting, and build checks before submitting changes. Fix failures introduced by changes.
- Follow existing project conventions when these standards do not specify a rule. Do not introduce new libraries, architectural patterns, or dependencies without team approval.

For AI coding assistants: Apply these standards to all generated or modified code. Inspect the existing project structure and configuration before making changes. Do not reformat unrelated files or introduce unnecessary refactoring. If a requirement conflicts with the existing configuration or is ambiguous, follow established project conventions and identify the conflict for the team rather than silently inventing a new rule.

For Further Reference

Rules of React.

ReactJS Coding Guidelines.

React Style Guide

An Opinionated Guide to React.js: Best Practices and Conventions

JavaScript and React/JSX Style Guides.